Semantic tagging

T3000 Haystack & Brick Auto-Tagging + MCP — Design Document · built Nov 13, 2026

T3000 Haystack & Brick Auto-Tagging + MCP — Design Document

1. Overview

1.1 Why

T3000 devices expose raw BACnet points: `Device 1001, AI-5, name="OAT", value=72.3`. Humans and LLMs cannot understand what this point represents without additional context. Vendor platforms (JCI OpenBlue, Honeywell Forge, Siemens Building X) each provide their own proprietary semantic layers, locked inside their ecosystems. Independent MSIs running mixed-vendor portfolios have no standard way to make their building data AI-readable.

1.2 Purpose

Add a semantic tagging layer on top of T3000's existing point data that:

- Automatically assigns Haystack tags (`[outside, air, temp]`) and Brick classes (`Outside_Air_Temperature_Sensor`) to every device point
- Exposes tagged data to LLM agents via MCP, enabling natural language queries like "Why is AHU-3 not cooling?"
- Keeps all logic local (no cloud dependency, no credentials egress)

1.3 Concepts Used

Concept	Source	Role
Haystack	Project Haystack (2011)	Flat tag vocabulary — `point` , `sensor` , `outside` , `air` , `temp`
Brick	Brick Schema (2016, UC Berkeley)	Formal class hierarchy — `Outside_Air_Temperature_Sensor isA Sensor isA Point`
YAML Rules	brick-bacnet-mcp (Yves Habchy, 2026)	Regex patterns mapping point names → tags + classes
MCP	Model Context Protocol (Anthropic, 2024)	JSON-RPC/stdio protocol for LLM agent tool access

1.4 How It Integrates With T3000

T3000 v2 already has Haystack tagging (manual assignment, basic auto-tagging). V3 adds:

TEXT

v2 (current)		v3
Hardcoded auto_tag_point()	→	Rules table + regex engine
Haystack tags only	→	+ Brick class per point
POST /api/haystack/rebuild	→	/api/haystack/auto-tagging/run
No MCP	→	MCP server (7 tools over stdio)
No rules management UI	→	Auto-Tagging & MCP page (3 tabs)

1.5 End-to-End Flow

TEXT

```
1. DEVICE SYNC
   C++ FFI → INPUTS/OUTPUTS/VARIABLES tables updated

2. AUTO-TAGGING (three-step engine)
   Step 1 — Range rules: lookup (point_type, digital_analog, Range_Field) → baseline tag
   Step 2 — Haystack regex: eval_rules(label, units) → extra tags (INSERT OR IGNORE)
   Step 3 — Brick regex: eval_rules(label, units) → brick_class (overrides range)
   → Write tags to HAYSTACK_POINT_TAGS (auto_assigned=1)
   → Write brick_class to HAYSTACK_POINT_BRICK_CLASS

3. WEB UI
   Auto-Tagging & MCP page → manage rules, run tagging, view MCP status
   Inputs/Outputs/Variables pages → Tags column shows tags + Brick badge

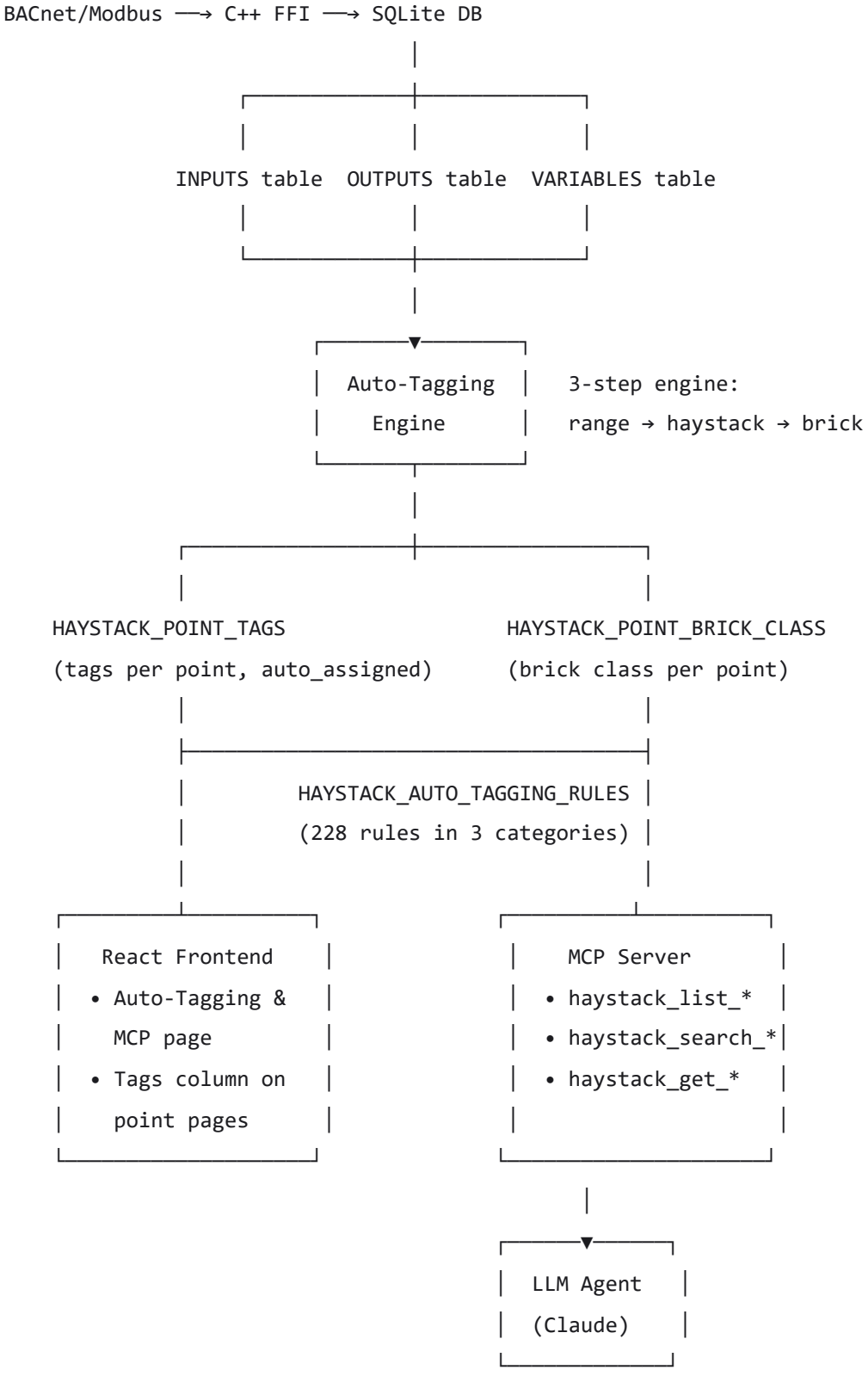
4. MCP SERVER
   Tools query HAYSTACK_POINT_TAGS + HAYSTACK_POINT_BRICK_CLASS + point tables
   Returns semantic-tagged data via JSON-RPC/stdio

5. LLM AGENT
   User: "What's the outside air temperature?"
   Agent → get_tagged_topology(filter_brick="Outside_Air_Temperature_Sensor")
        → Returns: 72.3°F
```

---

2. Architecture Diagram

TEXT



3. Database Schema

3.1 `HAYSTACK_POINT_TAGS` — tag assignments (1 row per tag per point)

```SQL
CREATE TABLE HAYSTACK_POINT_TAGS (
    serial_number  INTEGER NOT NULL,
    point_type     TEXT NOT NULL,
    point_index    TEXT NOT NULL,
    point_id       TEXT NOT NULL,
    tag_name       TEXT NOT NULL,
    auto_assigned  INTEGER NOT NULL DEFAULT 0,  -- 1=auto-tagged, 0=manual
    PRIMARY KEY (serial_number, point_type, point_index, tag_name)
);
```

- `auto_assigned` added by m20260716; default 0 preserves existing manual tags
- Auto-tagging INSERTs with `auto_assigned=1`
- Reset deletes `WHERE auto_assigned=1` — manual tags survive
- Old `brick_class` column dropped in m20260716

3.2 `HAYSTACK_POINT_BRICK_CLASS` — brick classification (1 row per point)

```SQL
CREATE TABLE HAYSTACK_POINT_BRICK_CLASS (
    serial_number INTEGER NOT NULL,
    point_type    TEXT NOT NULL,
    point_index   INTEGER NOT NULL,
    brick_class   TEXT NOT NULL,
    PRIMARY KEY (serial_number, point_type, point_index)
);
```

- Separate from `HAYSTACK_POINT_TAGS` — brick_class is point-level, not tag-level
- Replaces old `__brick_class__` marker row pattern from v3.0
- Preview joins with `LEFT JOIN HAYSTACK_POINT_BRICK_CLASS`

3.3 `HAYSTACK_AUTO_TAGGING_RULES` — rule definitions

Three categories share one table (haystack, brick, range):

SQL

```sql
CREATE TABLE HAYSTACK_AUTO_TAGGING_RULES (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    rule_name       TEXT NOT NULL UNIQUE,
    category        TEXT NOT NULL CHECK(category IN ('haystack','brick','range')),

    -- Regex rules (haystack / brick)
    pattern         TEXT,           -- regex (NULL for range rules)
    units           TEXT,           -- units filter or display unit
    object_types    TEXT,           -- object type filter or range category

    -- Range rules (range)
    point_type      TEXT,           -- 'INPUT'|'OUTPUT'|'VARIABLE' (NULL for regex)
    digital_analog INTEGER,         -- 0=digital, 1=analog (NULL for regex)
    range_value     INTEGER,        -- matches Range_Field (NULL for regex)

    -- Output (all categories)
    haystack_tags TEXT,             -- comma-separated
    brick_class    TEXT,            -- Brick class name
    haystack_kind TEXT,             -- Haystack kind tag
    haystack_unit TEXT,             -- Normalized Haystack unit

    -- Control
    enabled        INTEGER NOT NULL DEFAULT 1,
    priority       INTEGER NOT NULL DEFAULT 0,
    created_at     TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at     TEXT DEFAULT CURRENT_TIMESTAMP
);
```

Category usage matrix:

Column	haystack	brick	range
`pattern`	regex	regex	NULL
`units`	optional filter	optional filter	display unit (e.g. "Amps")
`object_types`	optional filter	optional filter	range category (e.g. "Temperature")
`point_type`	NULL	NULL	INPUT/OUTPUT/VARIABLE
`digital_analog`	NULL	NULL	0 or 1
`range_value`	NULL	NULL	0–104
`haystack_tags`	comma-separated	NULL	comma-separated
`brick_class`	NULL	class name	class name

3.4 Seed Data

Category	Count	Source
`haystack` (regex)	31	brick-bacnet-mcp YAML
`brick` (regex)	37	brick-bacnet-mcp YAML
`range` (metadata)	~160	Frontend rangeData.ts
Total	**~228**	

Rule examples:

```
rule_name:    brick:oat          category: brick
pattern:      (?i)(oat\|outside[_ ]?air[_ ]?temp)
brick_class:  Outside_Air_Temperature_Sensor

rule_name:    hs:oat             category: haystack
pattern:      (?i)(oat\|outside[_ ]?air[_ ]?temp)
haystack_tags: point,sensor,outside,air,temp

rule_name:    range:in-ana-3     category: range
point_type:   INPUT              digital_analog: 1    range_value: 3
haystack_tags: point,sensor,air,temp
brick_class:  Temperature_Sensor
units:        Deg.C
```

---

4. Rule Engine

File: `api/src/haystack/auto_tagging_service.rs`

4.1 Evaluation — Three-Step Flow

Auto-tagging runs in three sequential steps per point. Range rules apply first (metadata-based), then regex rules refine.

TEXT

```
For each point (row from INPUTS/OUTPUTS/VARIABLES):
  Read: SerialNumber, point_index, Full_Label, Label, Units,
        Digital_Analog, Range_Field

  display_label = Full_Label || Label || ""

  ┌── STEP 1: Range Rules ──────────────────────────┐
  │  SELECT * FROM HAYSTACK_AUTO_TAGGING_RULES      │
  │  WHERE category='range' AND enabled=1           │
  │    AND point_type=? AND digital_analog=?        │
  │    AND range_value=?                            │
  │                                                 │
  │  → INSERT tags into HAYSTACK_POINT_TAGS         │
  │    (auto_assigned=1, INSERT OR IGNORE on PK)    │
  │  → INSERT OR REPLACE brick_class into           │
  │    HAYSTACK_POINT_BRICK_CLASS                   │
  └─────────────────────────────────────────────────┘

  ┌── STEP 2: Haystack Regex Rules ─────────────────┐
  │  eval_rules(display_label, units, None,         │
  │             haystack_rules)                     │
  │  → INSERT OR IGNORE extra tags (auto_assigned=1)│
  │  → IGNORE prevents overwriting Step 1 tags      │
  └─────────────────────────────────────────────────┘

  ┌── STEP 3: Brick Regex Rules ────────────────────┐
  │  eval_rules(display_label, units, None,         │
  │             brick_rules)                        │
  │  → INSERT OR REPLACE brick_class                │
  │  → REPLACE = regex overrides range-based class  │
  └─────────────────────────────────────────────────┘

  tagged++ if any step produced output
```

4.2 `eval_rules()` — Regex Matching

RUST

```
fn eval_rules(label, units, object_type, rules) -> Option<&CompiledRule> {
    for rule in rules {  // priority-ordered, first match wins
        if !rule.regex.is_match(label)      { continue; }
        if rule.units_filter is set
           && point.units not in filter     { continue; }
        if rule.object_types_filter is set
           && point.type not in filter      { continue; }
        return Some(rule);
    }
    None
}
```

4.3 Triggers

Trigger	Code Path
Manual "Run Auto-Tag"	`POST /api/haystack/auto-tagging/run` → `run_auto_tagging()`
"Preview" button	`POST /api/haystack/auto-tagging/preview` → `preview_auto_tagging()`
"Reset Tags" button	`POST /api/haystack/auto-tagging/reset` → `reset_auto_tags()`

5. API Routes

5.1 Existing (Unchanged from v2)

All v2 routes remain under `/api/haystack/` — tags CRUD, point-tags read/write, tag-tree, replace-tag, sync. See v2-haystack-current-implementation.md §3.

5.2 Auto-Tagging Routes

File: `api/src/haystack/auto_tagging_routes.rs`

Route	Method	Input	Output
`/api/haystack/auto-tagging/rules`	GET	Query: `?category=haystack|brick|range`	`{ rules: TaggingRule[], total: N }`
`/api/haystack/auto-tagging/rules`	POST	`{ rule_name, category, pattern?, units?, object_types?, haystack_tags?, brick_class?, haystack_kind?, haystack_unit?, point_type?, digital_analog?, range_value?, priority? }`	`{ id, message }`
`/api/haystack/auto-tagging/rules/:id`	PUT	Partial rule fields	`{ message }`
`/api/haystack/auto-tagging/rules/:id`	DELETE	—	`{ message }`
`/api/haystack/auto-tagging/rules/:id/toggle`	POST	—	`{ enabled, message }`
`/api/haystack/auto-tagging/run`	POST	`{ serialNumbers: [i32] }`	`{ success, message, tagged: N, matches: TagMatch[] }`
`/api/haystack/auto-tagging/preview`	POST	`{ serialNumbers: [i32] }`	`{ matches: TagMatch[], total: N }`
`/api/haystack/auto-tagging/reset`	POST	`{ serialNumbers: [i32] }`	`{ success, message }`

6. MCP Server

File: `api/src/haystack/mcp.rs`

6.1 Transport

- Protocol: JSON-RPC 2.0 over stdio (MCP specification)
- Startup: Embedded in Rust binary, launched with `--mcp` flag
- Auto-discovery: Registered as `t3000` MCP server

6.2 Tools

Tool	Description
`t3000_haystack_list_tags`	List all available Haystack tags
`t3000_haystack_get_point_tags`	Get tags for a specific point
`t3000_haystack_search_points`	Search points by tag or brick class filter
`t3000_haystack_auto_tag`	Trigger auto-tagging on selected devices
`t3000_haystack_preview_tags`	Preview auto-tagging results without applying
`t3000_haystack_list_rules`	List all auto-tagging rules
`t3000_haystack_get_brick_class`	Get brick class for points (queries `HAYSTACK_POINT_BRICK_CLASS`)

6.3 Tool Output Format

JSON

```
// list_objects response
{
  "device": { "serial_number": 237219, "name": "T3-XX-ESP111" },
  "points": [
    {
      "point_id": "dev237219.in5",
      "point_type": "INPUT",
      "point_index": "5",
      "label": "OAT Sensor",
      "value": 72.3,
      "units": "degF",
      "haystack_tags": ["point", "sensor", "outside", "air", "temp"],
      "brick_class": "Outside_Air_Temperature_Sensor",
      "haystack_kind": "Number",
      "haystack_unit": "°F"
    }
  ]
}
```

6.4 Agent Configuration

JSON

```
// Claude Desktop: claude_desktop_config.json
{
  "mcpServers": {
    "t3000": {
      "command": "t3000-webview",
      "args": ["--mcp"]
    }
  }
}
```

---

7. Frontend

7.1 New Page — Auto-Tagging & MCP

Property	Value
URL	`/t3000/auto-tagging-mcp`
Component	`src/t3-react/features/haystack/pages/AutoTaggingMcpPage.tsx`
Store	Extends `haystackStore.ts` with `autoTaggingRules`, `mcpStatus` state
Menu	Haystack → Auto-Tagging & MCP

Tabs:

Tab	Content
Rules	CRUD table for all rules (haystack, brick, range), filter by category
Run Auto-Tag	Device selection, preview, run, and reset auto-tagging
MCP Server	Tool reference, connection info for LLM agents

Persistent status bar across all tabs:

TEXT

```
Auto-Tagging [ON ●——]  Device: T3-XX-ESP111  Tagged: 156/192 (81%)
[Run Auto-Tagging]  [View Coverage Report]
```

7.2 Existing Pages — Additions

Page	Change	Component
Inputs page	Tags column: add `brick_class` badge	`TagsColumnCell.tsx`
Outputs page	Same	Same
Variables page	Same	Same
Tags Drawer	Add Brick class dropdown	Tag drawer component

7.3 Menu & Routes

Menu config: `src/t3-react/config/menuConfig.ts`

```text
Haystack ▼
  ├── Standard Tags       (/t3000/haystack-tags)
  ├── Custom Tags         (/t3000/custom-tags)
  └── Auto-Tagging & MCP  (/t3000/auto-tagging-mcp)
```

Routes: `src/t3-react/app/router/routes.ts`

```text
/t3000/auto-tagging-mcp  →  AutoTaggingMcpPage
```

---

8. Rust Code Structure

TEXT

```
api/src/haystack/
  ├── mod.rs
  ├── tags_service.rs
  ├── tags_routes.rs
  ├── auto_tagging_service.rs       ← rule engine + rules CRUD + eval_rules
  ├── auto_tagging_routes.rs        ← run/preview/reset + rules CRUD endpoints
  └── mcp.rs                        ← MCP server (7 tools, JSON-RPC/stdio)

api/migration/src/
  ├── m20260715_add_auto_tagging_rules.rs
  ├── m20260716_add_point_brick_class_table.rs
  └── m20260717_add_range_rules.rs  (to be created)

src/t3-react/features/haystack/pages/
  └── AutoTaggingMcpPage.tsx        ← Rules / Run Auto-Tag / MCP Server tabs
```

---

9. Migrations

m20260715 — Original rules + brick_class column

- Creates `HAYSTACK_AUTO_TAGGING_RULES` with 68 seed rules (37 brick + 31 haystack)
- Adds `brick_class` column to `HAYSTACK_POINT_TAGS`

m20260716 — Point brick class table + auto_assigned

1. Creates `HAYSTACK_POINT_BRICK_CLASS` table
2. Migrates old `__brick_class__` marker rows → new table, then deletes them
3. Adds `auto_assigned` column to `HAYSTACK_POINT_TAGS` (DEFAULT 0)
4. Drops old `brick_class` column from `HAYSTACK_POINT_TAGS`

m20260717 — Range rules

1. Adds `point_type`, `digital_analog`, `range_value` columns to rules table
2. Drops/recreates CHECK constraint to include `'range'` category
3. Seeds ~160 range rules from frontend `rangeData.ts`

---

10. Related Documents

Document	Link
Haystack v2 Implementation Guide	v2-haystack-current-implementation.md
Haystack v1 (deprecated)	v1-haystack-legacy-single-table.md
Brick Schema	https://brickschema.org/
brick-bacnet-mcp (upstream)	https://github.com/Yveshby27/brick-bacnet-mcp